Republic of the Philippines
POLYTECHNIC UNIVERSITY OF THE PHILIPPINES
SAN PEDRO CAMPUS
PUP, Main Road, Barangay United Bayanihan, City of San Pedro, Laguna

HealthBridge Clinic Management System (HBCMS) Roles and Privileges

Activity 4 Progress Documentation

1. Introduction

This documentation presents the database setup, role configuration, privilege assignment, and access control testing performed for Group04's HealthBridge Clinic Management System (HBCMS).

The purpose of this activity is to apply database security principles by creating appropriate database roles, assigning privileges based on their responsibilities, and verifying that unauthorized operations are properly restricted.

The database was configured locally using PostgreSQL 18.6. The PHP backend was also tested to verify that the application could successfully connect to the database using its assigned database account.

2. Project Overview

Project Name: HealthBridge Clinic Management System (HBCMS)

Owning Group: Group04

Repository: https://github.com/RainneQuintab/GRP03_PROJECTDB

Database Name: grp03_projectdb

Technology Used:

- Frontend: HTML, CSS, and JavaScript
- Backend: PHP
- Database: PostgreSQL

- Local Server: XAMPP and Apache

The HealthBridge Clinic Management System is designed to manage clinic operations, including patient records, appointments, doctors, medical notes, prescriptions, billing, and staff accounts.

The application supports different user types, such as patients, staff, doctors, and administrators. Each user type has different responsibilities and access requirements.

3. Database Inspection

Before implementing access control, the project repository and database documentation were reviewed to understand the system structure and its database requirements.

The database was successfully created using PostgreSQL 18.6.

Database Name: grp03_projectdb

Schema: public

Extension: pgcrypto

The database schema and seed data were executed successfully.

3.1 Database Tables

The database contains 11 tables:

No.	Table Name
1	appointments
2	billing
3	deleted_records
4	doctors
5	medical_notes
6	patient_password_resets
7	patients

8	prescriptions
9	specialties
10	staff_password_resets
11	staff_users

3.2 Database Views

The database also contains two views:

1. appointment_details
2. visit_records

These views provide organized information for appointment and visit-related records.

3.3 Application Access Requirements

Based on the project documentation and backend source code, the application has the following access requirements:

Application Role	Main Responsibilities
Patient	Register, log in, request appointments, and view personal medical and billing records
Staff	Manage appointments, patient information, and billing records
Doctor	View assigned appointments and manage medical notes and prescriptions
Admin	Manage staff operations and access backup, restore, and permanent deletion functions

The application uses a shared PostgreSQL account for backend database operations. Individual application roles are checked by the PHP backend.

4. Role Creation and Configuration

To implement database access control, three PostgreSQL roles were created.

4.1 Created Database Roles

Role Name	Role Type	Description
hb_admin	NOLOGIN	Administrative group role with full table and sequence privileges
hb_staff	NOLOGIN	Group role for clinic operations
hb_app	LOGIN	Application database account that inherits privileges from hb_staff

4.2 Role Configuration

The following roles were created:

CREATE ROLE hb_admin NOLOGIN;
CREATE ROLE hb_staff NOLOGIN;
CREATE ROLE hb_app LOGIN INHERIT;

The hb_app account was assigned a password locally using PostgreSQL's password prompt. The actual password was not included in the SQL script or repository.

The application account was added as a member of the staff group role:

GRANT hb_staff TO hb_app;

This allows hb_app to inherit the permissions assigned to hb_staff.

4.3 Database and Schema Access

The required database connection and schema permissions were assigned:

GRANT CONNECT ON DATABASE grp03_projectdb

TO hb_admin, hb_staff, hb_app;

GRANT USAGE ON SCHEMA public

TO hb_admin, hb_staff, hb_app;

5. Privilege Assignment

Privileges were assigned based on the responsibilities of each database role.

5.1 Administrative Role

The hb_admin role was granted full privileges on existing tables and sequences.

```
GRANT ALL PRIVILEGES
ON ALL TABLES IN SCHEMA public
TO hb_admin;

GRANT ALL PRIVILEGES
ON ALL SEQUENCES IN SCHEMA public
TO hb_admin;
```

This allows the administrative role to perform table and sequence operations covered by those privileges.

5.2 Staff Role

The hb_staff role was granted the required permissions for clinic operations.

```
GRANT SELECT, INSERT, UPDATE
ON patients, appointments, medical_notes, prescriptions
TO hb_staff;

GRANT SELECT
ON doctors, specialties
TO hb_staff;

GRANT SELECT, UPDATE
ON staff_users
TO hb_staff;
```

Additional permissions were assigned to password reset records:

```
GRANT SELECT, INSERT, UPDATE
ON staff_password_resets, patient_password_resets
TO hb_staff;
```

Sequence permissions were also granted:

```
GRANT USAGE, SELECT
ON ALL SEQUENCES IN SCHEMA public
```

TO hb_staff;

5.3 Delete and Backup Permissions

The staff role was granted delete permissions on selected clinical tables:

GRANT DELETE
ON patients, appointments, medical_notes, prescriptions
TO hb_staff;

The backup records table was assigned the following permissions:

GRANT SELECT, INSERT, UPDATE, DELETE
ON deleted_records
TO hb_staff;

The required sequence permission was also granted:

GRANT USAGE, SELECT
ON SEQUENCE deleted_records_record_id_seq
TO hb_staff;

5.4 View Permissions

During testing, the application account initially could not access the two database views.

The issue was resolved by granting SELECT permission to hb_staff:

GRANT SELECT
ON appointment_details, visit_records
TO hb_staff;

5.5 Default Privileges

Default privileges were configured so that future tables created by the postgres role in the public schema would automatically grant all table privileges to hb_admin.

ALTER DEFAULT PRIVILEGES
FOR ROLE postgres
IN SCHEMA public
GRANT ALL PRIVILEGES ON TABLES TO hb_admin;

The configuration was verified through PostgreSQL's pg_default_acl catalog.

6. Access Control Testing

After creating the roles and assigning privileges, several tests were performed using the hb_app account.

The purpose was to confirm that permitted operations worked and restricted operations were denied.

6.1 Allowed Operations

The following operations were successfully tested:

Test	Expected Result	Result
Connect to grp03_projectdb	Allowed	Passed
SELECT from patients from patients	Allowed	Passed
SELECT from appointments	Allowed	Passed
SELECT from doctors	Allowed	Passed
SELECT from billing	Allowed	Passed
SELECT from appointment_details	Allowed	Passed
SELECT from visit_records	Allowed	Passed
Use assigned sequences	Allowed	Passed

The following record counts were returned during testing:

- Patients: 2
- Appointments: 2
- Doctors: 3
- Billing: 0
- Appointment Details: 2
- Visit Records: 2

6.2 Denied Operations

The following operations were tested to verify that unauthorized actions were restricted.

Test	Expected Result	Result
CREATE TABLE in public schema	Denied	Passed
DELETE from patient_password_resets	Denied	Passed
CREATE DATABASE	Denied	Passed
CREATE ROLE	Denied	Passed

All four restricted operations were successfully denied.

6.3 Administrative Privilege Verification

The privileges assigned to hb_admin were checked using PostgreSQL privilege verification functions.
The following permissions were confirmed:

- SELECT on patients
- INSERT on patients
- DELETE on patients
- USAGE on the patients sequence

All checked privileges returned true.

7. Issues and Limitations

Several issues and limitations were identified during database setup and access control testing.

7.1 Missing View Permissions

Issue: The application account initially received a permission error when accessing appointment_details and visit_records.

Resolution: SELECT privileges were granted to hb_staff on both views.

Result: Both views became accessible through hb_app.

7.2 Shared Application Database Account

Issue: The PHP backend uses one database account, hb_app, which inherits the permissions of hb_staff.

This means PostgreSQL cannot independently distinguish patient, doctor, staff, and admin operations when they all use the same database account.

Recommendation: Consider using separate database accounts or database roles for different application responsibilities if stronger database-level separation is required.

7.3 Administrative Role Ownership

Issue: The hb_admin role has full table and sequence privileges, but it is not the owner of the existing database objects.

Some ownership-level operations, such as altering or dropping objects, may still require the actual object owner or additional configuration.

Recommendation: Review whether administrative ownership or additional administrative privileges are required.

7.4 Group Number and Database Prefix Difference

Issue: The project documentation identifies the group as Group04, but the repository and database use the GRP03 prefix.

Resolution: The original database and repository names were retained. Activity 4 documentation was placed under db-access-control/Group04/.

8. Evidence and Repository

The SQL script and supporting screenshots for the HealthBridge access-control implementation are stored in the group's Car Rental Management System repository under db-access-control/Group04/.

Repository links:

- **Repository:** https://github.com/123airabels/Car_Rental_Management_System
- **UniSync access-control folder:** https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group04

- **SQL script:** https://github.com/123airabels/Car_Rental_Management_System/blob/main/db-access-control/Group04/grp04_access_control.sql
- **Evidence folder:** https://github.com/123airabels/Car_Rental_Management_System/tree/main/db-access-control/Group04/evidence

8.1 Evidence Screenshots

Screenshot	Description
[01]_grp03_database_objects.png	Database tables and views
[02]_grp03_roles_memberships.png	Created roles and role membership
[03]_grp03_allowed_access.png	Successful allowed access tests
[04]_grp03_denied_access.png	Restricted operation tests
[05]_grp03_admin_privileges.png	Administrative privilege verification
[06]_grp03_admin_default_privileges.png	Default privilege configuration
[07]_grp03_php_database_connection.png	Successful PHP database connection

9. SQL Scripts Summary

The following SQL scripts and commands were used during database setup and access control implementation.

Script / Command	Purpose
schema.sql	Creates the database tables, indexes, and extension
seed.sql	Inserts initial sample records
update_reason.sql	Applies the appointment reason update
update_note_unique.sql	Applies the medical note uniqueness update

update_billing.sql	Creates and configures billing-related database objects
create_views.sql	Creates appointment and visit-related views
demo_accounts.sql	Inserts or updates demonstration account data
grp04_access_control.sql	Documents the Activity 4 role and privilege configuration

The original roles.sql was reviewed, but the roles were created manually so that the hb_app password could be configured securely without placing a real password in the repository.

The upgrade-only scripts for appointments, backup, and patient password reset were not executed because the database was created from a fresh schema.

10. Verification Results

The database configuration and access control implementation were verified through PostgreSQL and PHP testing.

10.1 Database Setup

- Database creation: Successful
- Schema execution: Successful
- Seed data execution: Successful
- Additional database updates: Successful
- Database objects verification: Successful

10.2 Role Configuration

- hb_admin created successfully.
- hb_staff created successfully.
- hb_app created successfully.
- Role membership configured successfully.
- Database and schema permissions assigned successfully.

10.3 Access Control

- Allowed SELECT operations: Passed
- View access: Passed after permission correction

- Sequence access: Passed
- Unauthorized CREATE TABLE: Denied
- Unauthorized DELETE: Denied
- Unauthorized CREATE DATABASE: Denied
- Unauthorized CREATE ROLE: Denied

10.4 PHP Database Connection

The PHP backend was configured to connect to PostgreSQL using the hb_app account.

The application endpoint was tested through:

http://localhost/GRP03_PROJECTDB/backend/get_doctors.php

The endpoint returned a successful JSON response containing three doctor records.

Result: PHP-to-PostgreSQL connectivity was successfully verified.

11. Issues and Fixes

The following issues and limitations were encountered during the UniSync database setup and access-control implementation.

Issue	Action Taken / Resolution	Result
Missing SELECT permission on database views	Granted SELECT on both views to hb_staff	Fixed
Application database connection needed configuration	Created local config.php based on config.sample.php	Fixed
Password placeholder in sample configuration	Replaced locally with the configured hb_app password	Fixed
Project folder was not yet available in XAMPP	Copied the project into C:\xampp\htdocs\GRP03_PROJECTDB	Fixed

Group number differs from database prefix	Retained original database naming and used Group04 for Activity 4 documentation	Documented

12. Feedback to the Owning Group

The feedback covered the following points:

1. The database was successfully created and configured locally.
2. The PHP backend successfully connected to PostgreSQL using hb_app.
3. SELECT permissions for appointment_details and visit_records were initially missing but were corrected during testing.
4. The project currently uses a shared application database account. Separate database roles may be considered for stronger database-level access separation.
5. The administrative role's ownership and administrative capabilities may need further review.
6. The Group04 and GRP03 naming difference was identified and documented.

13. AI Usage Declaration

AI assistance was used during the Activity 4 process to support database administration and documentation tasks.

The assistance included:

- Reviewing database documentation and SQL scripts.
- Understanding application roles and their database access requirements.
- Preparing PostgreSQL role and privilege statements.
- Identifying possible database permission issues.
- Explaining PostgreSQL access control and privilege verification.
- Assisting with allowed and denied access testing.
- Organizing test results and preparing documentation.

All database commands were executed and tested in the local PostgreSQL environment. The database configuration, application connection, and access control results were verified through actual command outputs and application testing.

AI assistance was used as a supporting tool. The final implementation and verification were performed in the local environment.

14. Conclusion

The Activity 4 database access control implementation for Group04's HealthBridge Clinic Management System was successfully performed.

The database was created and populated with sample data. Three database roles were configured, and privileges were assigned according to their intended responsibilities. Allowed and denied operations were tested to verify that the access restrictions worked as expected.

The initial missing view permissions were corrected, and the PHP backend successfully connected to PostgreSQL through the hb_app account.

Some limitations were also identified, particularly the use of a shared application database account and the ownership limitations of the administrative role. These were documented along with recommendations for improvement.

Overall, the database setup, role configuration, privilege assignment, and access control verification were completed for the project.